Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
(ФГАОУ ВО СПбПУ)
Институт компьютерных наук и кибербезопасности
Высшая школа компьютерных технологий и информационных систем

# ПРАКТИЧЕСКАЯ РАБОТА

По дисциплине «Технология разработки программного обеспечения»

Искусственная жизнь

Студент гр. 5140901/51501 Н.С.Пиюренко

подпись, дата инициалы и фамилия

Оценка выполненной студентом работы:

Руководитель,
д.т.н, профессор ВШКТиИС. Г.Ф.Малыхина

подпись, дата инициалы и фамилия

Санкт-Петербург – 2025

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО

Институт компьютерных наук и кибербезопасности

Высшая школа компьютерных технологий и информационных систем

# ЗАДАНИЕ

на выполнение практической работы по дисциплине

«Технология разработки программного обеспечения»

студенту Пиюренко Никите Сергеевичу группа: 5140901/51501

1. Тема работы: Создание программы на языке программирования Java, моделирующая искусственную жизнь

2. Срок сдачи студентом законченной работы: 07.10.2025

3. Исходные данные по работе: Клеточное поле: 2500 клеток (50 x 50); Количество волков равное 30; Количество зайцев равное 60; Количество растений равное 500; Кролики и волки видят друг друга на 2 клетки поля; у каждого животного есть энергия, которая тратится на передвижение по клеточному полю; волки охотятся на кроликов, тем самым восполняют свою энергию; кролики едят растения, тем самым восполняют свою энергию; Если волк попадает в зону видимости кролика, то последний убегает от первого; если энергия всех представителей класса животных заканчивается, то симуляция считается завершенной.

## СОДЕРЖАНИЕ

1 Общие сведения ..... 4
1.1 Идея программы ..... 4
1.2 Основные правила ..... 4
2 Реализация классов ..... 6
2.1 Класс Hare (Зайцы) ..... 6
2.2 Класс Wolf (Волки) ..... 7
2.3 Класс Flower (Цветы) ..... 8
3 Реализация игровой логики ..... 9
3.1 Инициализация поля (HareGameGUI.initializeField()) ..... 9
3.2 Размещение объектов (HareGameGUI.initializeGameObjects()) ..... 9
3.3 Алгоритм движения зайцев (HareGameGUI.moveHares()) ..... 11
3.4 Алгоритм движения волков (HareGameGUI.moveWolves()) ..... 13
3.5 Проверка столкновений (HareGameGUI.checkCollisions()) ..... 14
4 Реализация графического интерфейса ..... 16
4.1 Структура GUI ..... 16
4.2 Визуальное представление ..... 16
4.3 Система отображения статистики ..... 17
5 Особенности реализации ..... 18
5.1 Управление ходами ..... 18
5.2 Механика поиска объектов ..... 18
5.3 Обработка граничных условий ..... 19
6 Критерии окончания игры ..... 20
Приложение ..... 21

# 1 Общие сведения

## 1.1 Идея программы

Данный программный продукт представляет собой имитационную модель замкнутой экосистемы, функционирующей по заданным правилам. Основная идея заключается в визуализации и анализе взаимодействия между популяциями хищников (волков 30 особей), травоядных (зайцев 60 особей) и пищевого ресурса (цветов 500 штук). Моделирование построено на агентном подходе, где каждое существо (агент) обладает собственным состоянием (координаты, энергия) и поведением, определяемым его ролью в экосистеме.

Целью проекта является создание наглядного инструмента для наблюдения за динамикой популяций, где исход (выживание одной из популяций) является результат множества локальных взаимодействий, а не заранее запрограммированный сценарий.

Такие модели полезны для изучения основ экологии, теории игр и агентного моделирования.

## 1.2 Основные правила

Для обеспечения стабильности и предсказуемости симуляции была разработана система строгих правил виртуального мира:

- **Размер поля:** Пространство ограничено сеткой 50 x 50 клеток. Это создает условия ограниченности ресурсов и пространства, что является ключевым фактором в межвидовой конкуренции. Клеточная структура упрощает вычисления и визуализацию;
- **Радиус видимости:** Ограничение радиуса восприятия животных 2 клетками имитирует реальные ограничения живых существ. Это означает, что животные действуют на основе локальной, а не глобальной

информации, что делает их поведение более реалистичным и менее оптимальным;

- **Энергетическая модель:** Каждое действие (перемещение, поиск) требует затрат в 1 единицу энергии. Это вводит в систему универсальную "валюту" выживания. Существа должны постоянно находить пищу для поддержания жизнедеятельности, что создает постоянное давление естественного отбора.
- **Пищевая цепочка:**
    1) Цветы, будучи съеденными зайцем, восстанавливают его энергетический запас на +2 единицы;
    2) Зайцы, будучи съеденными волком, передают ему +2 единицы энергии, выступая в роли преобразователя энергии от растений к хищникам.
- **Условие завершения:** Игра заканчивается, когда одна из популяций животных полностью вымирает. Это моделирует экологическую катастрофу, вызванную нарушением баланса в пищевой цепи: либо хищники истребляют всех травоядных и умирают от голода, либо травоядные, избавившись от угрозы, теоретически могли бы бесконтрольно размножаться.

# 2 Реализация классов

## 2.1 Класс Hare (Зайцы)

Класс Hare является моделью травоядного существа. Его состояние описывается следующими полями:

- **x, y** — координаты расположения особи на игровом поле. Определяют ее позицию в двумерном пространстве;
- **energy** — текущий запас энергии. Начальное значение 10 единиц дает зайцу некоторый запас времени на поиск пищи после старта симуляции. Энергия является критическим параметром для выживания;
- **Score** — счетчик, учитывающий количество успешно собранных цветов. Этот параметр не влияет на жизнеспособность особи, но является полезной метрикой для статистики и анализа эффективности поведения;
- **Alive** — булевский флаг, указывающий на жизнеспособность объекта. "Мертвые" зайцы исключаются из всех процессов симуляции.

```
class Hare {  14 usages
    private int x;  3 usages
    private int y;  3 usages
    private int energy;  5 usages
    private int score;  3 usages
    private boolean alive;  3 usages

    public Hare(int x, int y) {  1 usage
        this.x = x;
        this.y = y;
        this.energy = 10;
        this.score = 0;
        this.alive = true;
    }
```

Рисунок 2.1.1 — Класс Hare

Поведение зайца инкапсулировано в методах:

- **move()** — определяет логику выбора следующей клетки для перемещения на основе состояния окружающей среды;

- **eatFlower()** — обрабатывает акт поедания цветка: увеличивает энергию, увеличивает счетчик и переводит цветок в состояние «собран»;
- **useEnergy()** — уменьшает запас энергии на 1 единицу при совершении действия. Вызывает проверку на смерть от голода;
- **die()** — изменяет статус alive на false, сигнализируя о «гибели» особи.

## 2.2 Класс Wolf (Волки)

Класс Wolf реализует модель хищника. Его структура полей аналогична классу Hare (x, y, energy, alive), что отражает общие свойства всех мобильных агентов в системе. Отсутствие поля score подчеркивает разницу в целях: для волка важнее выживание, а не накопление "очков".

```
class Wolf {  14 usages
    private int x;  3 usages
    private int y;  3 usages
    private int energy;  5 usages
    private boolean alive;  3 usages

    public Wolf(int x, int y) {  1 usage
        this.x = x;
        this.y = y;
        this.energy = 10;
        this.alive = true;
    }
```

Рисунок 2.2.1 — Класс Wolf

Ключевое поведенческое отличие заключается в методе eatHare(), который описывает взаимодействие с жертвой. При столкновении с зайцем волк восстанавливает энергию, а заяц устраняется из симуляции. Методы move(), useEnergy() и die() реализуют ту же логику, что и у зайцев, но алгоритм, заложенный в move(), принципиально иной и направлен на поиск добычи, а не убегание от угрозы.

## 2.3 Класс Flower (Цветы)

Класс Flower представляет статичный ресурс. Его состояние минимально:

- **x, y** — координаты на поле;
- **collected** — флаг, указывающий, был ли цветок собран зайцем.

```
class Flower {  9 usages
    private int x;  2 usages
    private int y;  2 usages
    private boolean collected;  3 usages

    public Flower(int x, int y) {  1 usage
        this.x = x;
        this.y = y;
        this.collected = false;
    }
```

Рисунок 2.3.1 — Класс Flower

# 3 Реализация игровой логики

## 3.1 Инициализация поля (HareGameGUI.initializeField())

Процедура инициализации создает абстрактное представление игрового мира в виде двумерного массива размером 50 x 50 элементов. Каждый элемент массива соответствует одной клетке на поле. Изначально все клетки заполняются символом-заполнителем ('.'), что означает пустую клетку. Этот массив служит основой для последующего размещения объектов и для отрисовки состояния мира в графическом интерфейсе.

```
private void initializeField() {  1 usage
    field = new char[FIELD_SIZE][FIELD_SIZE];
    for (int i = 0; i < FIELD_SIZE; i++) {
        for (int j = 0; j < FIELD_SIZE; j++) {
            field[i][j] = EMPTY;
        }
    }
}
```

Рисунок 3.1.1 — Инициализация поля

## 3.2 Размещение объектов (HareGameGUI.initializeGameObjects())

Размещение начальной популяции объектов является критически важным этапом, так как от него зависит стартовый баланс системы. Процесс выполняется в строгой последовательности для избежания конфликтов:

- **Размещение 60 зайцев:** генерируются случайные координаты. Если клетка свободна (содержит '.'), в нее помещается объект Hare, а в соответствующую ячейку массива поля записывается символ 'H'. Процесс повторяется, пока не будет создано все 60 особей;

```
private void initializeGameObjects() {  1 usage
    // Создаем 60 зайцев
    hares = new Hare[60];
    for (int i = 0; i < hares.length; i++) {
        int x, y;
        do {
            x = random.nextInt(FIELD_SIZE);
            y = random.nextInt(FIELD_SIZE);
        } while (field[y][x] != EMPTY);

        hares[i] = new Hare(x, y);
        field[y][x] = HARE;
    }
```

Рисунок 3.2.1 — Инициализация зайцев

- **Размещение 30 волков:** алгоритм аналогичен, но проверяется, чтобы клетка не была занята зайцем. В массив поля записывается символ 'W';\

```
// Создаем 30 волков
wolves = new Wolf[30];
for (int i = 0; i < wolves.length; i++) {
    int x, y;
    do {
        x = random.nextInt(FIELD_SIZE);
        y = random.nextInt(FIELD_SIZE);
    } while (field[y][x] != EMPTY);

    wolves[i] = new Wolf(x, y);
    field[y][x] = WOLF;
}
```

Рисунок 3.2.2 — Инициализация волков

- **Размещение 500 цветов:** заполняются оставшиеся свободные клетки объектами класса Flower с записью символа 'F' в массив. Большое количество цветов обеспечивает первоначальную обильную кормовую базу для зайцев.

```
    // Создаем 500 цветков
    flowers = new Flower[500];
    for (int i = 0; i < flowers.length; i++) {
        int x, y;
        do {
            x = random.nextInt(FIELD_SIZE);
            y = random.nextInt(FIELD_SIZE);
        } while (field[y][x] != EMPTY);

        flowers[i] = new Flower(x, y);
        field[y][x] = FLOWER;
    }
}
```

Рисунок 3.2.3 — Инициализация цветков

## 3.3 Алгоритм движения зайцев (HareGameGUI.moveHares())

Для каждого живого зайца на каждом ходу выполняется следующий алгоритм принятия решения:

- **Оценка угрозы:** сканируется область в радиусе 2 клеток. Если в пределах этой зоны обнаружен хотя бы один волк, приоритетом становится бегство. Алгоритм вычисляет направление от ближайшего хищника;

```
private void moveHares() {  2 usages
    for (Hare hare : hares) {
        if (hare != null && hare.isAlive()) {
            int currentX = hare.getX();
            int currentY = hare.getY();

            // Ищем ближайшего волка в радиусе видимости
            Wolf nearestWolf = findNearestWolf(currentX, currentY);

            int newX = currentX;
            int newY = currentY;

            if (nearestWolf != null) {
                // Убегаем от волка
                if (currentX < nearestWolf.getX()) newX--;
                else if (currentX > nearestWolf.getX()) newX++;

                if (currentY < nearestWolf.getY()) newY--;
                else if (currentY > nearestWolf.getY()) newY++;
            } else {
```

Рисунок 3.3.1 — Алгоритм принятия решений, оценка угрозы

- **Поиск пищи:** если непосредственной угрозы нет, заяц переходит к поиску цветов в том же радиусе. Если цветок найден, заяц движется в его направлении;

```
// Если волков нет поблизости, ищем ближайший цветок
Flower nearestFlower = findNearestFlower(currentX, currentY);

if (nearestFlower != null && !nearestFlower.isCollected()) {
    // Двигаемся к цветку
    if (currentX < nearestFlower.getX()) newX++;
    else if (currentX > nearestFlower.getX()) newX--;

    if (currentY < nearestFlower.getY()) newY++;
    else if (currentY > nearestFlower.getY()) newY--;
} else {
```

Рисунок 3.3.2 — Алгоритм принятия решений, поиск пищи

- **Случайное блуждание:** если в радиусе видимости нет ни угроз, ни целей, заяц совершает случайный ход в одном из 8 возможных направлений (по горизонтали, вертикали или диагонали);

```
        // Случайное движение, если цветков тоже нет
        int direction = random.nextInt( bound: 4);
        switch (direction) {
            case 0: newX++; break; // вправо
            case 1: newX--; break; // влево
            case 2: newY++; break; // вниз
            case 3: newY--; break; // вверх
        }
    }
}
```

Рисунок 3.3.3 — Алгоритм принятия решений, случайное движение

- **Валидация хода:** проверяется, не выходит ли новая позиция за границы поля;

```
// Проверяем валидность хода
if (isValidMove(newX, newY)) {
    // Проверяем, есть ли на новой позиции цветок ДО движения
    boolean wasFlower = field[newY][newX] == FLOWER;

    // Двигаем зайца
    moveAnimal(hare, newX, newY);
```

Рисунок 3.3.4 — Алгоритм принятия решений, проверка валидности хода

- **Взаимодействие с ресурсом:** если заяц переместился на клетку с цветком, автоматически вызывается метод eatFlower();
- **Энергетический обмен:** независимо от результата хода, заяц тратит 1 единицу энергии на совершение действия;

```
    // Если на новой позиции был цветок - съедаем его
    if (wasFlower) {
        hare.eatFlower();
        totalFlowersCollected++;
        removeFlower(newX, newY);
    }
}
```

Рисунок 3.3.5 — Алгоритм принятия решений, энергетический обмен

- **Проверка жизнеспособности:** если энергия падает до 0 или ниже, вызывается метод die().

```
            // Тратим энергию
            if (!hare.useEnergy( amount: 1)) {
                hare.die();
                field[hare.getY()][hare.getX()] = EMPTY;
            }
        }
    }
}
```

Рисунок 3.3.6 — Алгоритм принятия решений, проверка жизнеспособности

## 3.4 Алгоритм движения волков (HareGameGUI.moveWolves())

Алгоритм для волков отражает их роль вершины пищевой цепи:

- **Поиск добычи:** в радиусе 2 клеток ищется ближайший заяц;

```
private void moveWolves() {  2 usages
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            int currentX = wolf.getX();
            int currentY = wolf.getY();

            // Ищем ближайшего зайца в радиусе видимости
            Hare nearestHare = findNearestHare(currentX, currentY);

            int newX = currentX;
            int newY = currentY;
```

Рисунок 3.4.1 — Алгоритм принятия решений, поиск зайца

- **Преследование:** если заяц обнаружен, волк движется прямо к нему;

```
if (nearestHare != null && nearestHare.isAlive()) {
    // Преследуем зайца
    if (currentX < nearestHare.getX()) newX++;
    else if (currentX > nearestHare.getX()) newX--;

    if (currentY < nearestHare.getY()) newY++;
    else if (currentY > nearestHare.getY()) newY--;
} else {
```

Рисунок 3.4.2 — Алгоритм принятия решений, приследование зайца

- **Случайное движение:** в отсутствие добычи волк перемещается случайным образом, исследуя территорию;

```
    // Случайное движение, если зайцев нет поблизости
    int direction = random.nextInt( bound: 4);
    switch (direction) {
        case 0: newX++; break;
        case 1: newX--; break;
        case 2: newY++; break;
        case 3: newY--; break;
    }
}
```

Рисунок 3.4.3 — Алгоритм принятия решений, случайное движение

- **Валидация и энергия:** как и заяц, волк проверяет валидность хода и тратит 1 единицу энергии;
- **Смерть от голода:** при истощении энергетических запасов волк умирает.

```
            // Проверяем валидность хода и двигаем волка
            if (isValidMove(newX, newY) && (field[newY][newX] == EMPTY || field[newY][newX] == FLOWER || field[newY][newX] == HARE)) {
                moveAnimal(wolf, newX, newY);
            }
            // Тратим энергию волка
            if (!wolf.useEnergy( amount: 1)) {
                wolf.die();
                field[wolf.getY()][wolf.getX()] = EMPTY;
            }
        }
    }
}
```

Рисунок 3.4.4 — Алгоритм принятия решений, проверка жизнеспособности

## 3.5 Проверка столкновений (HareGameGUI.checkCollisions())

Данный метод реализует механику хищничества. После того как все волки и зайцы совершили свои ходы, система проверяет все возможные пары "волк-

заяц" на предмет нахождения на одной клетке. Если такое столкновение обнаружено:

- Объект зайца помечается как мертвый (hare.die()).
- Волк, находящийся на той же клетке, получает энергетическую подпитку (wolf.eatHare()), что увеличивает его шансы на выживание и дальнейшую охоту.
- Символ зайца 'H' на игровом поле заменяется на символ волка 'W'.

```
private void checkCollisions() {  2 usages
    // Проверяем, поймали ли волки зайцев
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            for (Hare hare : hares) {
                if (hare != null && hare.isAlive() &&
                        hare.getX() == wolf.getX() && hare.getY() == wolf.getY()) {
                    hare.die();
                    wolf.eatHare();
                    field[hare.getY()][hare.getX()] = WOLF;
                }
            }
        }
    }
}
```

Рисунок 3.5.1 — Проверка столкновений волка и зайца

# 4 Реализация графического интерфейса

## 4.1 Структура GUI

Графический интерфейс пользователя построен на основе библиотеки Swing и организован по иерархическому принципу:

```
JFrame (главное окно)
├── JPanel (заголовок с номером хода)
├── JPanel (основная панель)
│   ├── JScrollPane (игровое поле 50×50)
│   └── JTextArea (статистика)
└── JPanel (кнопка "Следующий ход")
```

## 4.2 Визуальное представление

Для обеспечения максимальной наглядности каждому типу объекта присвоен уникальный цветовой код и символ:

- **Зайцы ('H'):** отображаются на светло-зеленом фоне;
- **Волки ('W'):** отображаются на светло-красном фоне;
- **Цветы ('F'):** отображаются на светло-желтом фоне;
- **Пустые клетки ('.'):** имеют белый фон.

Такое кодирование позволяет пользователю мгновенно оценить общую ситуацию на поле: распределение популяций, скопления ресурсов и зоны взаимодействия.

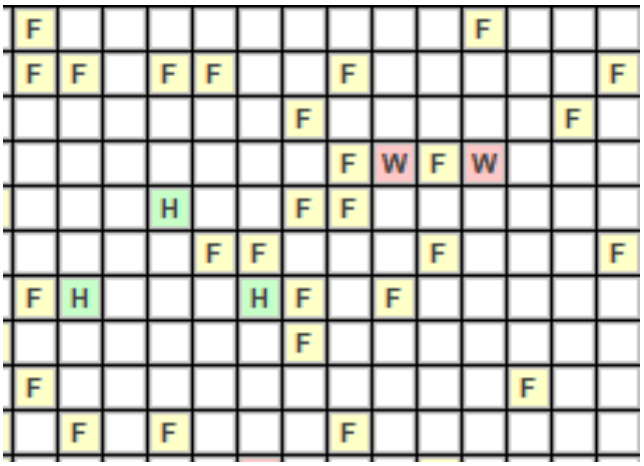


Рисунок 4.2.1 — Визуальное представление объектов

## 4.3 Система отображения статистики

JTextArea в правой части окна выводит актуальную количественную информацию о состоянии экосистемы:

- **Популяционная динамика:** Текущее количество живых зайцев и волков;
- **Ресурсы:** Общее количество собранных и оставшихся цветков;
- **Витальность популяций:** Средний уровень энергии среди живых зайцев и волков. Низкий средний показатель энергии может служить индикатором грядущего вымирания;
- **Легенда:** Постоянное напоминание о значении символов, используемых на поле.

Эта информация необходима для анализа долгосрочных тенденций в симуляции.

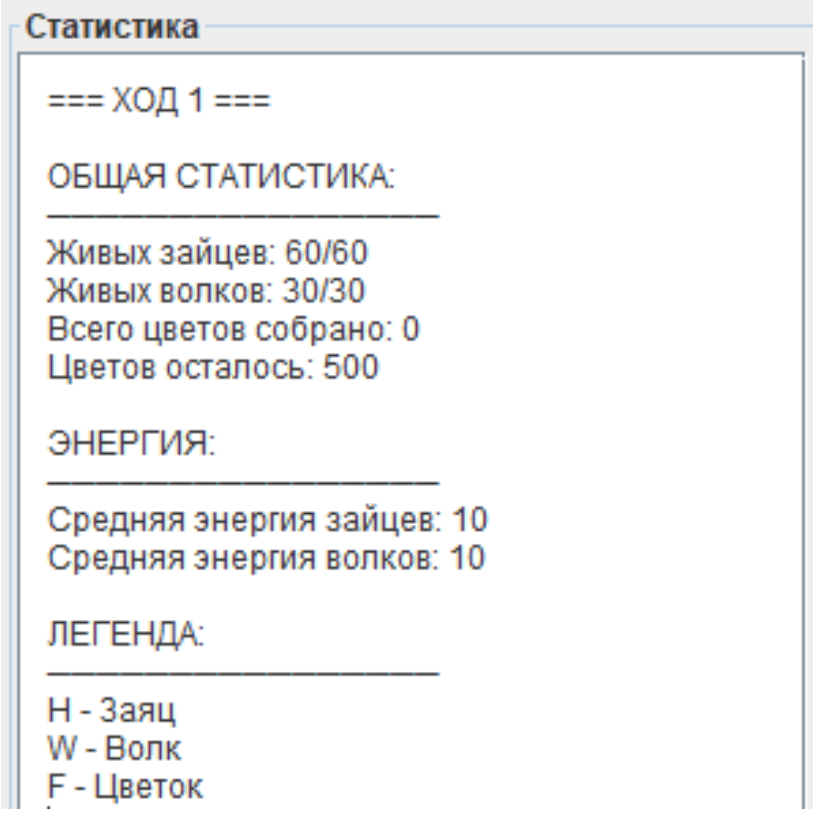


Рисунок 4.3.1 — Блок «Статистика»

# 5 Особенности реализации

## 5.1 Управление ходами

Реализация пошагового режима с помощью кнопки «Следующий ход» является ключевым элементом. Она позволяет пользователю внимательно наблюдать за каждым шагом симуляции и анализировать причины и следствия каждого действия агентов.

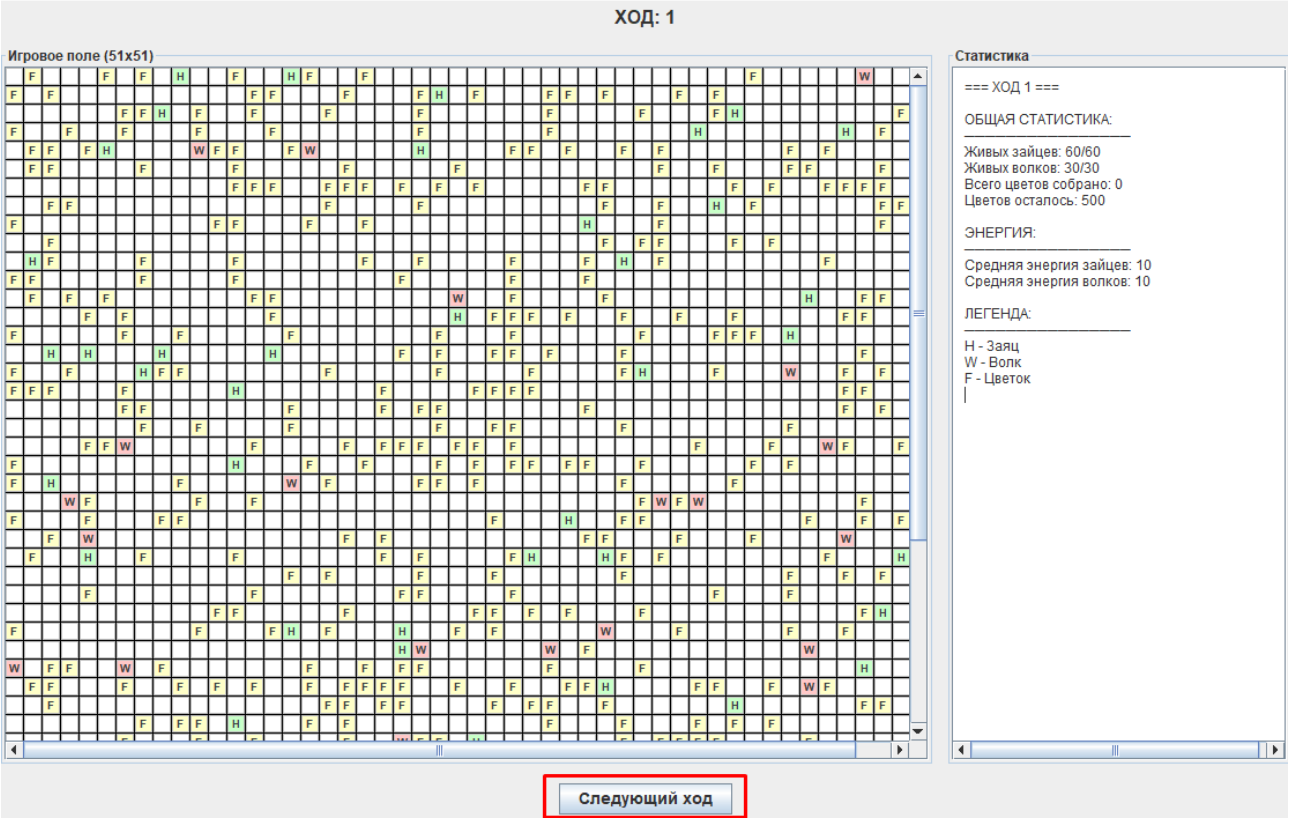


Рисунок 5.1.1 — Интерфейс программы с кнопкой «Следующий ход»

## 5.2 Механика поиска объектов

Для определения расстояния между объектами используется Манхэттенское расстояние (расстояние городских кварталов), которое вычисляется как сумма модулей разностей координат по осям X и Y: $d = |x1 - x2| + |y1 - y2|$. Этот метрика была выбрана вместо Евклидовой расстояния по следующим причинам:

- **Вычислительная эффективность:** отсутствие операции извлечения квадратного корня ускоряет расчеты, что критично при большом количестве агентов;
- **Соответствие сетке:** движение агентов происходит по клеточной сетке, где перемещение по диагонали считается за один шаг, что интуитивно ближе к Манхэттенскому расстоянию.

## 5.3 Обработка граничных условий

Корректная обработка границ и коллизий — залог устойчивой работы симуляции:

- **Границы поля:** перед каждым перемещением проверяется, не приведет ли ход к выходу за пределы массива (координаты < 0 или >= 50). В случае нарушения границы ход блокируется, и агент остается на месте (или генерируется новое направление);
- **Размещение объектов:** при инициализации гарантируется, что на одной клетке может находиться только один объект. Это предотвращает наложение символов на поле и логические ошибки (например, рождение волка внутри зайца);
- **Коллизии при движении:** механизм проверки столкновений checkCollisions() обрабатывает ситуации, когда два агента оказываются на одной клетке после выполнения всех ходов, разрешая конфликт в пользу хищника.

# 6 Критерии окончания игры

Симуляция подходит к логическому завершению, когда одна из популяций активных агентов перестает существовать, что означает нарушение базового принципа функционирования пищевой цепи:

- **Победа волков (экологический коллапс):** если все зайцы истреблены, программа выводит соответствующее сообщение. Волки, оставшись без источника пищи, обречены на вымирание, но игра фиксирует их тактическую победу, приведшую, однако, к уничтожению всей экосистемы;

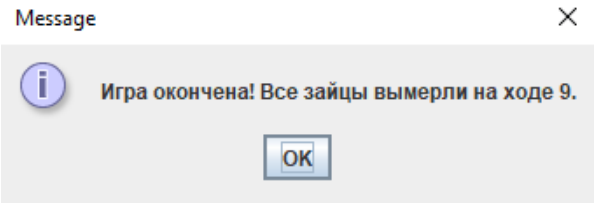


Рисунок 6.1 — Предупреждающее сообщение

- **Победа зайцев (исчезновение хищников):** если все волки погибли (вероятнее всего, от голода из-за недостаточной эффективности охоты), программа сообщает о победе зайцев. В этой ситуации популяция зайцев теоретически может бесконтрольно потреблять ресурсы, но в данной модели их размножение не реализовано, поэтому симуляция просто останавливается.

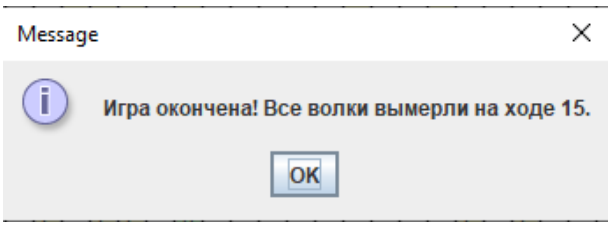


Рисунок 6.2 — Предупреждающее сообщение

Эти условия позволяют определить результат каждого запуска и проанализировать, какие начальные условия или изменения в поведении агентов приводят к тому или иному исходу.

## Приложение

Программный код на языке Java, моделирующая искусственную жизнь 3 типов объектов:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class HareGameGUI {
    private static final int FIELD_SIZE = 51;
    private static final int VISIBILITY_RANGE = 2;
    private static final char EMPTY = '.';
    private static final char HARE = 'H';
    private static final char WOLF = 'W';
    private static final char FLOWER = 'F';

    private char[][] field;
    private Hare[] hares;
    private Wolf[] wolves;
    private Flower[] flowers;
    private Random random;
    private int totalFlowersCollected;
    private int turn;

    // GUI компоненты
    private JFrame frame;
    private JPanel gridPanel;
    private JTextArea statsArea;
    private JButton nextTurnButton;
    private JLabel turnLabel;
    private JLabel[][] cellLabels;

    public HareGameGUI() {
        random = new Random();
        totalFlowersCollected = 0;
        turn = 1;
        initializeField();
        initializeGameObjects();
        createGUI();
    }

    private void initializeField() {
        field = new char[FIELD_SIZE][FIELD_SIZE];
        for (int i = 0; i < FIELD_SIZE; i++) {
            for (int j = 0; j < FIELD_SIZE; j++) {
                field[i][j] = EMPTY;
            }
        }
    }

    private void initializeGameObjects() {
        // Создаем 60 зайцев
        hares = new Hare[60];
        for (int i = 0; i < hares.length; i++) {
            int x, y;
```

```
        do {
            x = random.nextInt(FIELD_SIZE);
            y = random.nextInt(FIELD_SIZE);
        } while (field[y][x] != EMPTY);

        hares[i] = new Hare(x, y);
        field[y][x] = HARE;
    }

    // Создаем 30 волков
    wolves = new Wolf[30];
    for (int i = 0; i < wolves.length; i++) {
        int x, y;
        do {
            x = random.nextInt(FIELD_SIZE);
            y = random.nextInt(FIELD_SIZE);
        } while (field[y][x] != EMPTY);

        wolves[i] = new Wolf(x, y);
        field[y][x] = WOLF;
    }

    // Создаем 500 цветков
    flowers = new Flower[500];
    for (int i = 0; i < flowers.length; i++) {
        int x, y;
        do {
            x = random.nextInt(FIELD_SIZE);
            y = random.nextInt(FIELD_SIZE);
        } while (field[y][x] != EMPTY);

        flowers[i] = new Flower(x, y);
        field[y][x] = FLOWER;
    }
}

private void createGUI() {
    frame = new JFrame("Симуляция зайцев и волков");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLayout(new BorderLayout(10, 10));

    // Панель заголовка
    JPanel headerPanel = new JPanel();
    turnLabel = new JLabel("ХОД: 1");
    turnLabel.setFont(new Font("Arial", Font.BOLD, 16));
    headerPanel.add(turnLabel);

    // Основная панель с полем и статистикой
    JPanel mainPanel = new JPanel(new BorderLayout(10, 10));

    // Поле игры в виде сетки
    JPanel fieldPanel = new JPanel(new BorderLayout());
    fieldPanel.setBorder(BorderFactory.createTitledBorder("Игровое поле (50x50)"));

    gridPanel = new JPanel(new GridLayout(FIELD_SIZE, FIELD_SIZE, 1, 1));
    gridPanel.setBackground(Color.BLACK);
    cellLabels = new JLabel[FIELD_SIZE][FIELD_SIZE];

    // Создаем ячейки сетки
    for (int y = 0; y < FIELD_SIZE; y++) {
        for (int x = 0; x < FIELD_SIZE; x++) {
```

```
            JLabel cell = new JLabel("", SwingConstants.CENTER);
            cell.setPreferredSize(new Dimension(15, 15));
            cell.setMinimumSize(new Dimension(15, 15));
            cell.setOpaque(true);
            cell.setBackground(Color.WHITE);
            cell.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY, 1));
            cell.setFont(new Font("Arial", Font.BOLD, 10));

            cellLabels[y][x] = cell;
            gridPanel.add(cell);
        }
    }

    JScrollPane gridScroll = new JScrollPane(gridPanel);
    gridScroll.setPreferredSize(new Dimension(800, 600));
    fieldPanel.add(gridScroll, BorderLayout.CENTER);

    // Панель статистики
    JPanel statsPanel = new JPanel(new BorderLayout());
    statsPanel.setBorder(BorderFactory.createTitledBorder("Статистика"));
    statsPanel.setPreferredSize(new Dimension(300, 0));

    statsArea = new JTextArea(20, 25);
    statsArea.setFont(new Font("Arial", Font.PLAIN, 12));
    statsArea.setEditable(false);
    statsArea.setMargin(new Insets(10, 10, 10, 10));
    JScrollPane statsScroll = new JScrollPane(statsArea);
    statsPanel.add(statsScroll, BorderLayout.CENTER);

    mainPanel.add(fieldPanel, BorderLayout.CENTER);
    mainPanel.add(statsPanel, BorderLayout.EAST);

    // Панель кнопок
    JPanel buttonPanel = new JPanel();
    nextTurnButton = new JButton("Следующий ход");
    nextTurnButton.setFont(new Font("Arial", Font.BOLD, 14));
    nextTurnButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            nextTurn();
        }
    });

    buttonPanel.add(nextTurnButton);

    frame.add(headerPanel, BorderLayout.NORTH);
    frame.add(mainPanel, BorderLayout.CENTER);
    frame.add(buttonPanel, BorderLayout.SOUTH);

    frame.pack();
    frame.setExtendedState(JFrame.MAXIMIZED_BOTH);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);

    updateDisplay();
}

private void nextTurn() {
    turnLabel.setText("ХОД: " + turn);

    // Двигаем всех зайцев
```

```
        moveHares();

        // Двигаем всех волков
        moveWolves();

        // Проверяем столкновения
        checkCollisions();

        // Обновляем отображение
        updateDisplay();

        // Проверяем окончание игры
        if (isGameOver()) {
            String message;
            if (areAllHaresDead()) {
                message = "Игра окончена! Все зайцы вымерли на ходе " + turn + ".";
            } else {
                message = "Игра окончена! Все волки вымерли на ходе " + turn + ".";
            }
            JOptionPane.showMessageDialog(frame, message);
            nextTurnButton.setEnabled(false);
        }

        turn++;
    }

    private void autoPlay(int turns) {
        nextTurnButton.setEnabled(false);

        new Thread(() -> {
            for (int i = 0; i < turns; i++) {
                if (isGameOver()) break;

                SwingUtilities.invokeLater(() -> {
                    turnLabel.setText("ХОД: " + turn);
                    moveHares();
                    moveWolves();
                    checkCollisions();
                    updateDisplay();
                    turn++;
                });

                try {
                    Thread.sleep(500); // Задержка для визуализации
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }

            SwingUtilities.invokeLater(() -> {
                nextTurnButton.setEnabled(true);
                if (isGameOver()) {
                    String message;
                    if (areAllHaresDead()) {
                        message = "Игра окончена! Все зайцы вымерли на ходе " + (turn-1)
+ ".";
                    } else {
                        message = "Игра окончена! Все волки вымерли на ходе " + (turn-1)
+ ".";
                    }
                    JOptionPane.showMessageDialog(frame, message);
```

```
                nextTurnButton.setEnabled(false);
            }
        });
    }).start();
}

private void moveHares() {
    for (Hare hare : hares) {
        if (hare != null && hare.isAlive()) {
            int currentX = hare.getX();
            int currentY = hare.getY();

            // Ищем ближайшего волка в радиусе видимости
            Wolf nearestWolf = findNearestWolf(currentX, currentY);

            int newX = currentX;
            int newY = currentY;

            if (nearestWolf != null) {
                // Убегаем от волка
                if (currentX < nearestWolf.getX()) newX--;
                else if (currentX > nearestWolf.getX()) newX++;

                if (currentY < nearestWolf.getY()) newY--;
                else if (currentY > nearestWolf.getY()) newY++;
            } else {
                // Если волков нет поблизости, ищем ближайший цветок
                Flower nearestFlower = findNearestFlower(currentX, currentY);

                if (nearestFlower != null && !nearestFlower.isCollected()) {
                    // Двигаемся к цветку
                    if (currentX < nearestFlower.getX()) newX++;
                    else if (currentX > nearestFlower.getX()) newX--;

                    if (currentY < nearestFlower.getY()) newY++;
                    else if (currentY > nearestFlower.getY()) newY--;
                } else {
                    // Случайное движение, если цветков тоже нет
                    int direction = random.nextInt(4);
                    switch (direction) {
                        case 0: newX++; break; // вправо
                        case 1: newX--; break; // влево
                        case 2: newY++; break; // вниз
                        case 3: newY--; break; // вверх
                    }
                }
            }

            // Проверяем валидность хода
            if (isValidMove(newX, newY)) {
                // Проверяем, есть ли на новой позиции цветок ДО движения
                boolean wasFlower = field[newY][newX] == FLOWER;

                // Двигаем зайца
                moveAnimal(hare, newX, newY);

                // Если на новой позиции был цветок - съедаем его
                if (wasFlower) {
                    hare.eatFlower();
                    totalFlowersCollected++;
                    removeFlower(newX, newY);
```

```
                }
            }

            // Тратим энергию
            if (!hare.useEnergy(1)) {
                hare.die();
                field[hare.getY()][hare.getX()] = EMPTY;
            }
        }
    }
}

private void moveWolves() {
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            int currentX = wolf.getX();
            int currentY = wolf.getY();

            // Ищем ближайшего зайца в радиусе видимости
            Hare nearestHare = findNearestHare(currentX, currentY);

            int newX = currentX;
            int newY = currentY;

            if (nearestHare != null && nearestHare.isAlive()) {
                // Преследуем зайца
                if (currentX < nearestHare.getX()) newX++;
                else if (currentX > nearestHare.getX()) newX--;

                if (currentY < nearestHare.getY()) newY++;
                else if (currentY > nearestHare.getY()) newY--;
            } else {
                // Случайное движение, если зайцев нет поблизости
                int direction = random.nextInt(4);
                switch (direction) {
                    case 0: newX++; break;
                    case 1: newX--; break;
                    case 2: newY++; break;
                    case 3: newY--; break;
                }
            }

            // Проверяем валидность хода и двигаем волка
            if (isValidMove(newX, newY) && (field[newY][newX] == EMPTY ||
field[newY][newX] == FLOWER || field[newY][newX] == HARE)) {
                moveAnimal(wolf, newX, newY);
            }

            // Тратим энергию волка
            if (!wolf.useEnergy(1)) {
                wolf.die();
                field[wolf.getY()][wolf.getX()] = EMPTY;
            }
        }
    }
}

private Wolf findNearestWolf(int x, int y) {
    Wolf nearest = null;
    int minDistance = VISIBILITY_RANGE + 1;
```

```
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            int distance = Math.abs(wolf.getX() - x) + Math.abs(wolf.getY() - y);
            if (distance <= VISIBILITY_RANGE && distance < minDistance) {
                minDistance = distance;
                nearest = wolf;
            }
        }
    }

    return nearest;
}

private Hare findNearestHare(int x, int y) {
    Hare nearest = null;
    int minDistance = VISIBILITY_RANGE + 1;

    for (Hare hare : hares) {
        if (hare != null && hare.isAlive()) {
            int distance = Math.abs(hare.getX() - x) + Math.abs(hare.getY() - y);
            if (distance <= VISIBILITY_RANGE && distance < minDistance) {
                minDistance = distance;
                nearest = hare;
            }
        }
    }

    return nearest;
}

private Flower findNearestFlower(int x, int y) {
    Flower nearest = null;
    int minDistance = VISIBILITY_RANGE + 1;

    for (Flower flower : flowers) {
        if (flower != null && !flower.isCollected()) {
            int distance = Math.abs(flower.getX() - x) + Math.abs(flower.getY() - y);
            if (distance <= VISIBILITY_RANGE && distance < minDistance) {
                minDistance = distance;
                nearest = flower;
            }
        }
    }

    return nearest;
}

private void moveAnimal(Object animal, int newX, int newY) {
    if (animal instanceof Hare) {
        Hare hare = (Hare) animal;
        field[hare.getY()][hare.getX()] = EMPTY;
        hare.move(newX, newY);
        field[newY][newX] = HARE;
    } else if (animal instanceof Wolf) {
        Wolf wolf = (Wolf) animal;
        field[wolf.getY()][wolf.getX()] = EMPTY;
        wolf.move(newX, newY);
        field[newY][newX] = WOLF;
    }
}
```

```
private void removeFlower(int x, int y) {
    for (Flower flower : flowers) {
        if (flower != null && !flower.isCollected() &&
                flower.getX() == x && flower.getY() == y) {
            flower.collect();
            break;
        }
    }
}

private void checkCollisions() {
    // Проверяем, поймали ли волки зайцев
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            for (Hare hare : hares) {
                if (hare != null && hare.isAlive() &&
                        hare.getX() == wolf.getX() && hare.getY() == wolf.getY()) {
                    hare.die();
                    wolf.eatHare();
                    field[hare.getY()][hare.getX()] = WOLF;
                }
            }
        }
    }
}

private boolean isGameOver() {
    return areAllHaresDead() || areAllWolvesDead();
}

private boolean areAllHaresDead() {
    for (Hare hare : hares) {
        if (hare != null && hare.isAlive()) {
            return false;
        }
    }
    return true;
}

private boolean areAllWolvesDead() {
    for (Wolf wolf : wolves) {
        if (wolf != null && wolf.isAlive()) {
            return false;
        }
    }
    return true;
}

private boolean isValidMove(int x, int y) {
    return x >= 0 && x < FIELD_SIZE && y >= 0 && y < FIELD_SIZE;
}

private void updateDisplay() {
    updateGridDisplay();
    updateStatsDisplay();
}

private void updateGridDisplay() {
    for (int y = 0; y < FIELD_SIZE; y++) {
        for (int x = 0; x < FIELD_SIZE; x++) {
            JLabel cell = cellLabels[y][x];
```

```
                char cellContent = field[y][x];

                switch (cellContent) {
                    case HARE:
                        cell.setText("H");
                        cell.setBackground(new Color(200, 255, 200)); // Светло-зеленый
для зайцев
                        cell.setForeground(Color.DARK_GRAY);
                        break;
                    case WOLF:
                        cell.setText("W");
                        cell.setBackground(new Color(255, 200, 200)); // Светло-красный
для волков
                        cell.setForeground(Color.DARK_GRAY);
                        break;
                    case FLOWER:
                        cell.setText("F");
                        cell.setBackground(new Color(255, 255, 200)); // Светло-желтый
для цветов
                        cell.setForeground(Color.DARK_GRAY);
                        break;
                    case EMPTY:
                        cell.setText("");
                        cell.setBackground(Color.WHITE);
                        break;
                }

                // Добавляем всплывающую подсказку с координатами
                cell.setToolTipText("Координаты: (" + x + ", " + y + ")");
            }
        }
    }

    private void updateStatsDisplay() {
        int aliveHares = 0;
        int aliveWolves = 0;
        int totalHareEnergy = 0;
        int totalWolfEnergy = 0;
        int remainingFlowers = 0;

        // Считаем живых зайцев и их энергию
        for (Hare hare : hares) {
            if (hare != null && hare.isAlive()) {
                aliveHares++;
                totalHareEnergy += hare.getEnergy();
            }
        }

        // Считаем живых волков и их энергию
        for (Wolf wolf : wolves) {
            if (wolf != null && wolf.isAlive()) {
                aliveWolves++;
                totalWolfEnergy += wolf.getEnergy();
            }
        }

        // Считаем оставшиеся цветы
        for (Flower flower : flowers) {
            if (flower != null && !flower.isCollected()) {
                remainingFlowers++;
            }
```

```
        }

        StringBuilder stats = new StringBuilder();
        stats.append("=== ХОД ").append(turn).append(" ===\n\n");

        stats.append("ОБЩАЯ СТАТИСТИКА:\n");
        stats.append("────────────────\n");
        stats.append("Живых зайцев:
").append(aliveHares).append("/").append(hares.length).append("\n");
        stats.append("Живых волков:
").append(aliveWolves).append("/").append(wolves.length).append("\n");
        stats.append("Всего цветов собрано:
").append(totalFlowersCollected).append("\n");
        stats.append("Цветов осталось: ").append(remainingFlowers).append("\n\n");

        stats.append("ЭНЕРГИЯ:\n");
        stats.append("────────────────\n");
        stats.append("Средняя энергия зайцев: ").append(aliveHares > 0 ?
totalHareEnergy/aliveHares : 0).append("\n");
        stats.append("Средняя энергия волков: ").append(aliveWolves > 0 ?
totalWolfEnergy/aliveWolves : 0).append("\n\n");

        stats.append("ЛЕГЕНДА:\n");
        stats.append("────────────────\n");
        stats.append("H - Заяц \n");
        stats.append("W - Волк \n");
        stats.append("F - Цветок\n");

        statsArea.setText(stats.toString());
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new HareGameGUI();
            }
        });
    }
}

class Hare {
    private int x;
    private int y;
    private int energy;
    private int score;
    private boolean alive;

    public Hare(int x, int y) {
        this.x = x;
        this.y = y;
        this.energy = 10;
        this.score = 0;
        this.alive = true;
    }

    public void move(int newX, int newY) {
        this.x = newX;
        this.y = newY;
    }
```

```
    public void eatFlower() {
        energy += 2;
        score++;
    }

    public boolean useEnergy(int amount) {
        if (energy >= amount) {
            energy -= amount;
            return true;
        }
        return false;
    }

    public void die() {
        alive = false;
    }

    public int getX() { return x; }
    public int getY() { return y; }
    public int getEnergy() { return energy; }
    public int getScore() { return score; }
    public boolean isAlive() { return alive; }
}

class Wolf {
    private int x;
    private int y;
    private int energy;
    private boolean alive;

    public Wolf(int x, int y) {
        this.x = x;
        this.y = y;
        this.energy = 10;
        this.alive = true;
    }

    public void move(int newX, int newY) {
        this.x = newX;
        this.y = newY;
    }

    public void eatHare() {
        energy += 2;
    }

    public boolean useEnergy(int amount) {
        if (energy >= amount) {
            energy -= amount;
            return true;
        }
        return false;
    }

    public void die() {
        alive = false;
    }

    public int getX() { return x; }
    public int getY() { return y; }
    public int getEnergy() { return energy; }
```

```
    public boolean isAlive() { return alive; }
}

class Flower {
    private int x;
    private int y;
    private boolean collected;

    public Flower(int x, int y) {
        this.x = x;
        this.y = y;
        this.collected = false;
    }

    public void collect() {
        collected = true;
    }

    public int getX() { return x; }
    public int getY() { return y; }
    public boolean isCollected() { return collected; }
}
```